

Min Myint Moh Soe _ 6530262 _ Chapter 1_ Assignment

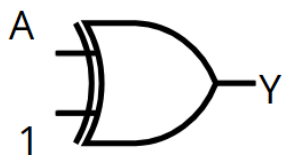
15. For N variables , there are $2^{(2^N)}$ truth tables.

16.

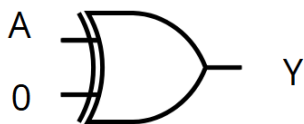
Truth Table for a 3-Input XOR Gate

A	B	C	A XOR B XOR C
0 ▾	0 ▾	0 ▾	0 ▾
0 ▾	0 ▾	1 ▾	1 ▾
0 ▾	1 ▾	0 ▾	1 ▾
0 ▾	1 ▾	1 ▾	0 ▾
1 ▾	0 ▾	0 ▾	1 ▾
1 ▾	0 ▾	1 ▾	0 ▾
1 ▾	1 ▾	0 ▾	0 ▾
1 ▾	1 ▾	1 ▾	1 ▾

17. NOT gate Using XOR Gates



BUFFER gate Using XOR Gates



Min Myint Moh Soe _ 6530262 _ Chapter 2_ Assignment

Exercise 1(W)

1. Sum-of-Products : $Y = A'B'C' + A'B'C + AB'C' + AB'C + AB'C'$

Exercise 2(W)

1. Sum-of-Products : $Y = A'B'C + A'BC' + AB'C + ABC$

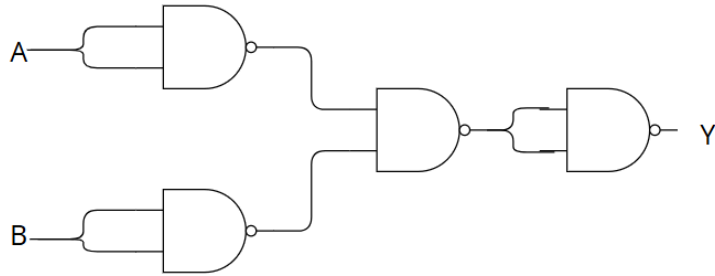
Exercise 3

1. Two-Input AND using NAND

$$AND(A, B) = NAND(NAND(A, B), NAND(A, B))$$

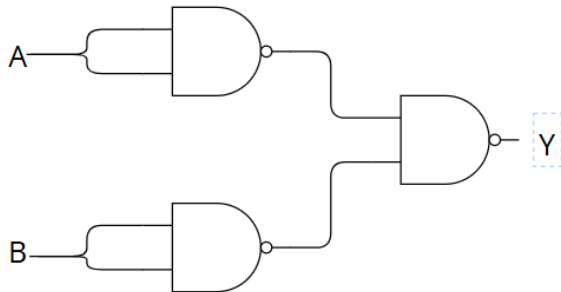
2. Two-Input NOR using NAND

$$NOR(A, B) = NAND(NAND(NAND(A, A), NAND(B, B)), NAND(NAND(A, A), NAND(B, B)))$$



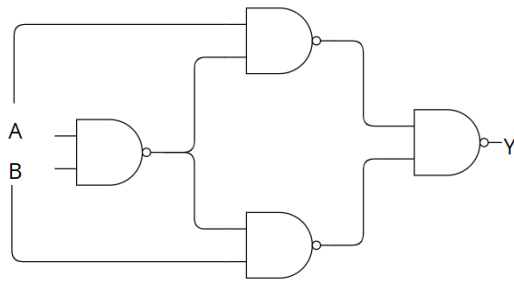
3. Two-Input OR using NAND

$$OR(A, B) = NAND(NAND(A, A), NAND(B, B))$$



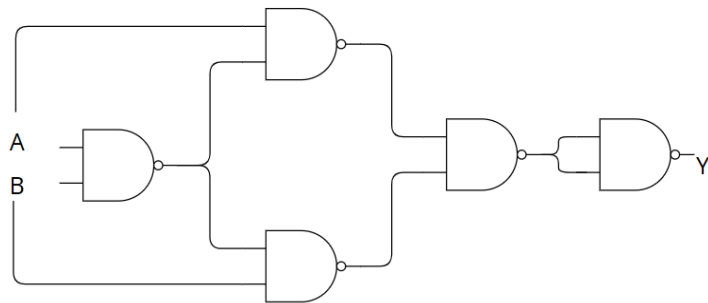
4. Two-Input XOR using NAND

$$XOR(A, B) = NAND(NAND(A, NAND(A, B)), NAND(B, NAND(A, B)))$$



5. Two-input XNOR using NAND

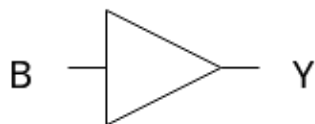
$$\text{XNOR}(A, B) = \text{NAND}(\text{XOR}(A, B), \text{XOR}(A, B))$$



Exercise 4

1. $Y = AB + A'BC + A'BC'$

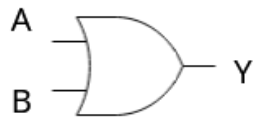
- $Y = AB + A'BC + A'BC'$
- $Y = AB + A'B(C + C')$
- $Y = AB + A'B$
- $Y = B(A + A')$
- $Y = B$



○

2. $Y = A + A'B + A'BC$

- $Y = A + A'B + A'BC$
- $Y = A + A'B(1 + C)$
- $Y = A + A'B$
- $Y = A + B$



○

3. $Y = (A + B + C)' + A'BC' + ABC'(W)$

4. $Y = (A + B)C + A'BC' + ABC'(W)$

Exercise 2.13

(a) $Y = AC + A'B'C$

- $Y = C(A + A'B')$
- $Y = C(A + B')$
- $Y = AC + B'C$

K-map

Y	AB				
		00	01	11	10
C	0	0	1	1	1
	1	0	1	0	1

(b) $Y = (AB)' + A'BC + (A' + C')(W)$

(c) $Y = A'B(CD)' + ABC' + AB(CD)' + ABD + A'BC'D + BC'D + A'(W)$

Exercise 2.14

(a) $Y = A'BC + A'BC'$

- $Y = A'B(C + C')$
- $Y = A'B$

Y	AB
---	----

C		00	01	11	10
	0	0	1	0	0
	1	0	1	0	0

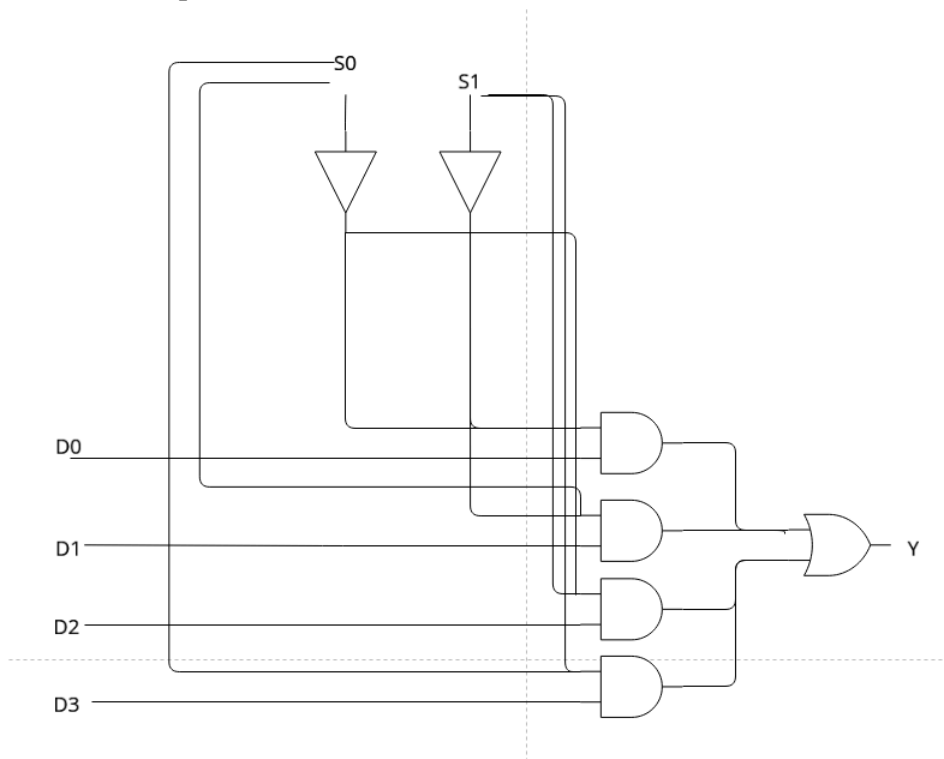
(b) $Y = ABC + AB$

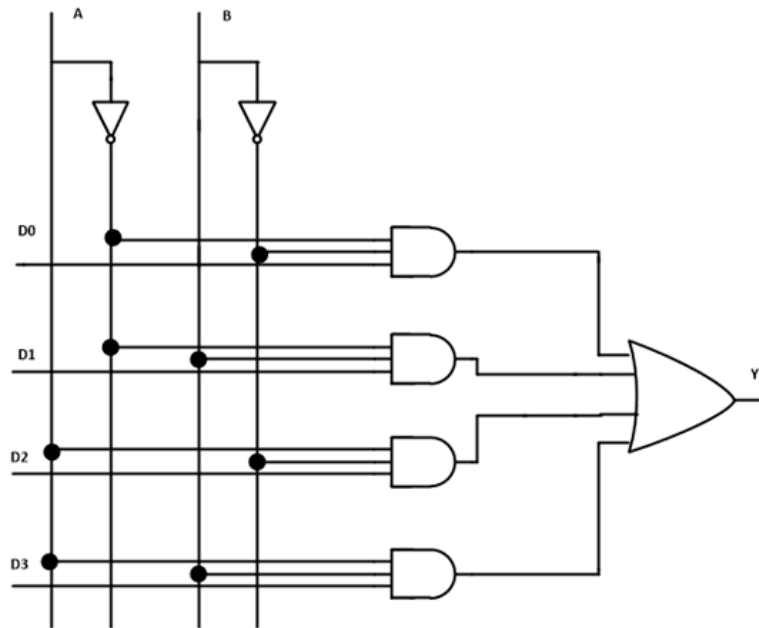
- $Y = AB(C+1)$
- $Y = AB$

Y	AB				
		00	01	11	10
C	0	0	0	1	0
	1	0	0	1	0

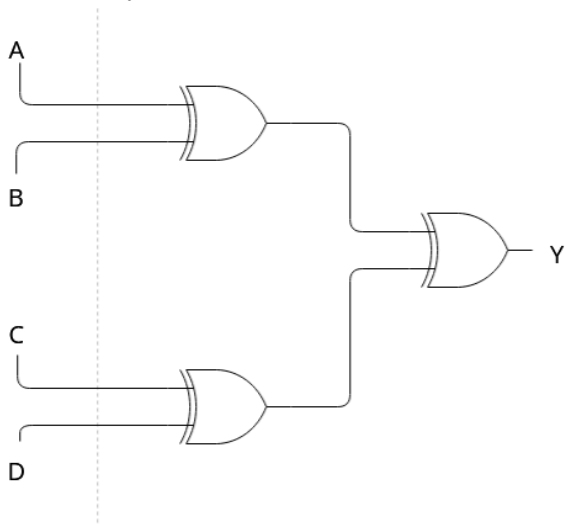
(c) $Y = AB(CD)' + ABC'D + (A+B+C+D)$

5. 4-line Multiplexer Schematic



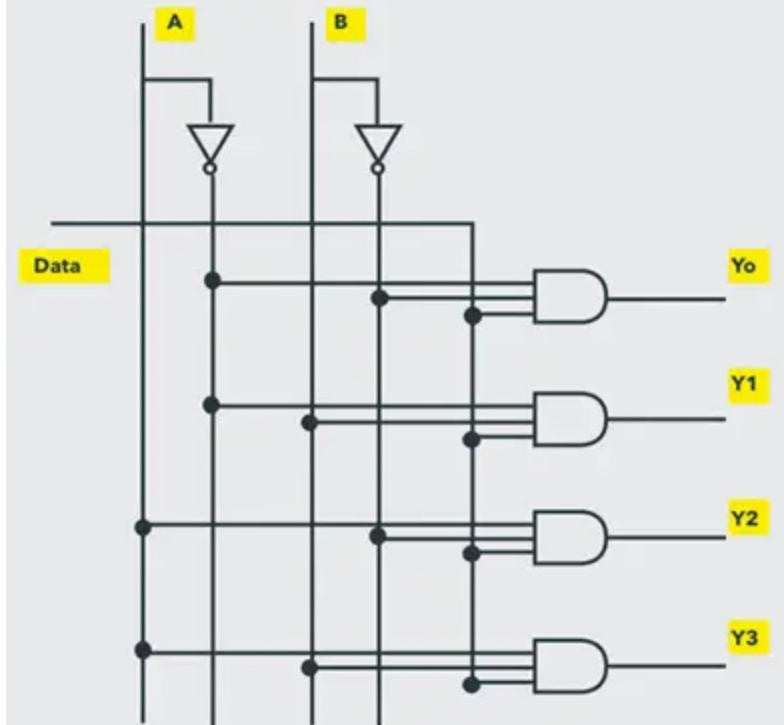


6. 4-bits Parity Checker

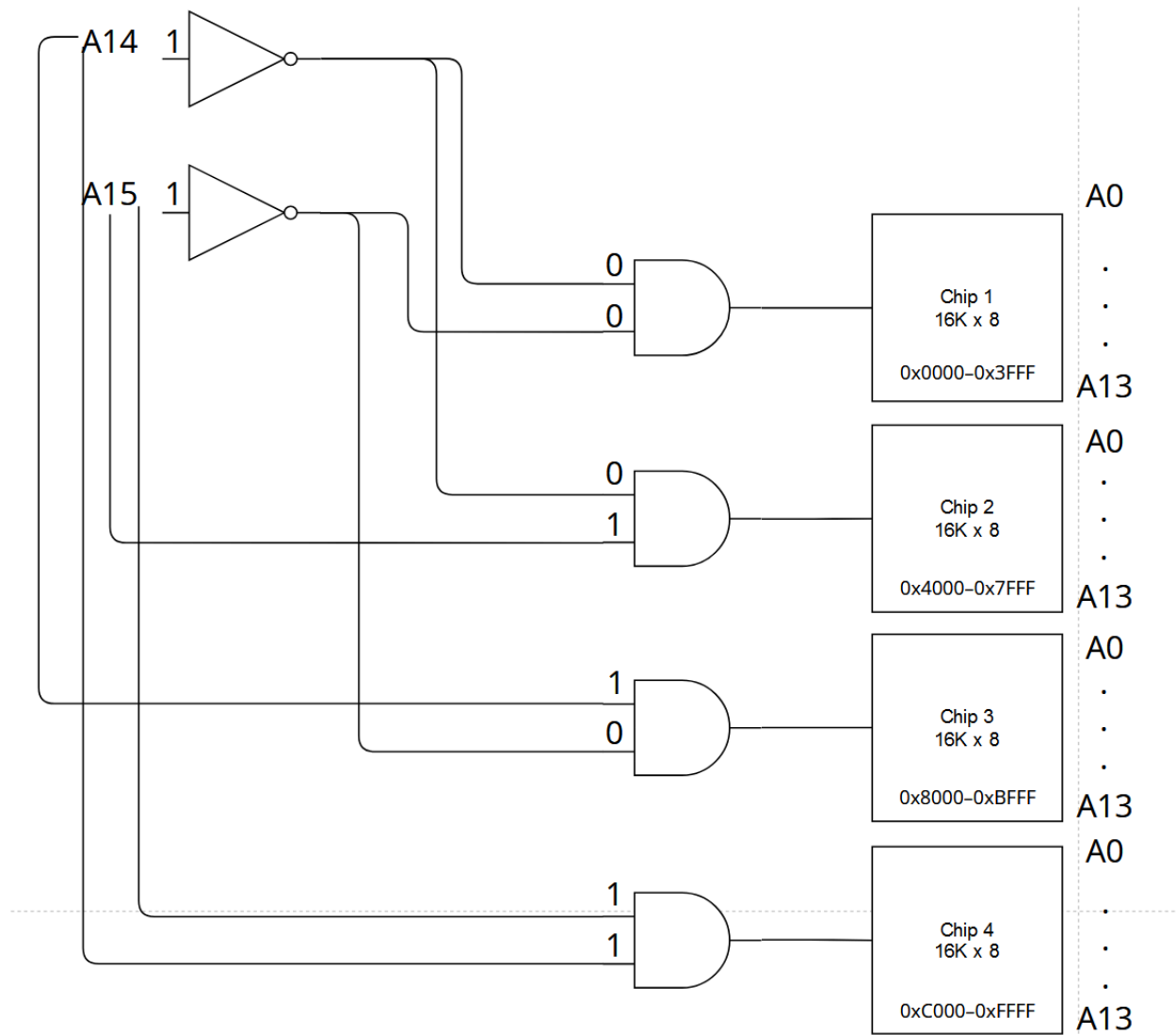


1:4 DEMUX

Logic Circuit Of 1-To-4 Demultiplexer



7. 4-line decoder and address ranges used by the decoder to select four memory chips (each with size 16K x 8).



Lower 14 lines: A0–A13 go into every chip to pick the individual byte in that chip.

- Why 14 lines? Because each chip is $16K = 2^{14}$ bytes.

Upper 2 lines: A14–A15 decide which of the four chips we want (because $2^2 = 4$).

- These two lines feed into a *2-to-4 line decoder*.

Definition: In memory math, “1K” = 1024 (which is 2^{10}), not 1000.

So: 16K bytes = 16,384 bytes in decimal.

2^{10} in hex is 0x400.

Doubling 0x400 is 0x800 (that’s 2^{11}).

Doubling 0x800 is 0x1000 (that’s 2^{12}), etc. Keep doubling until you get 0x4000.

16,384 bytes = 2^{14} = 0x4000

4 × 16K in decimal :

4×16,384=65,536(decimal)

65,536= 2^{16}

16-Bit Addresses: 2^{16} = 0x0000 to 0xFFFF

A 16-bit address bus means you have 16 binary lines (bits) for addresses. Each bit can be 0 or 1:

The lowest address (all bits = 0) is decimal 0 = 0x0000.

The highest address (all bits = 1) is decimal 65,535 = 0xFFFF.

Chip 0: A15..A14 = 00

- Start address = 0x0000
- End address = 0x3FFF
- Range = 0x0000–0x3FFF (16K total)

Chip 1: A15..A14 = 01

- Start address = 0x4000
- End address = 0x7FFF
- Range = 0x4000–0x7FFF (16K total)

Chip 2: A15..A14 = 10

- Start address = 0x8000
- End address = 0xBFFF
- Range = 0x8000–0xBFFF (16K total)

Chip 3: A15..A14 = 11

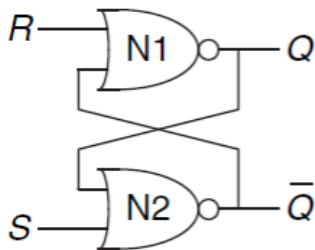
- Start address = 0xC000
- End address = 0xFFFF
- Range = 0xC000–0xFFFF (16K total)

6530262_MinMyintMohSoe_Assignment(3)

1. Difference between combinational and sequential logic circuits:

- **Combinational Logic Circuits:**
 - Outputs depend only on the current inputs.
 - No memory elements are involved.
- **Sequential Logic Circuits:**
 - Outputs depend on both current inputs and past inputs (stored in memory).
 - They include memory elements like flip-flops or latches.

2.



Truth Table

S	R	Q (Output)	$\bar{Q}$ (Complement of Output)
0 ▾	0 ▾	Previous Q	Previous $\bar{Q}$
0 ▾	1 ▾	0	1
1 ▾	0 ▾	1	0
1 ▾	1 ▾	Invalid	Invalid

The circuit in the figure is sequential logic. This circuit is called an SR (Set-Reset) Latch.

It is one of the simplest forms of sequential circuits, capable of storing one bit of information.

3.

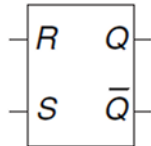
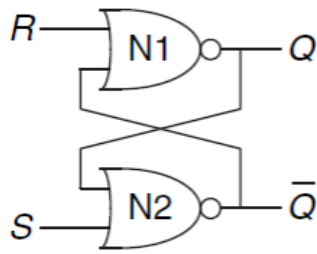
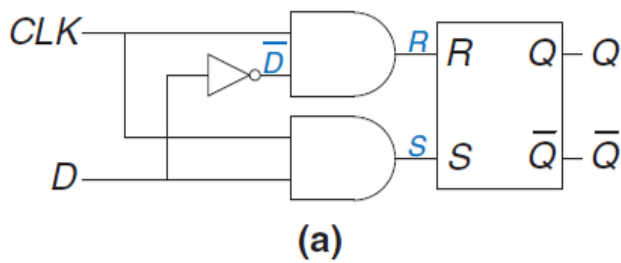


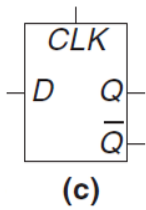
Figure 3.6 SR latch symbol

Truth Table Of S-R Latch

S	R	Q (Output)	$\bar{Q}$ (Complement of Output)
0 ▾	0 ▾	Previous Q	Previous $\bar{Q}$
0 ▾	1 ▾	0	1
1 ▾	0 ▾	1	0
1 ▾	1 ▾	Invalid	Invalid

D Latch

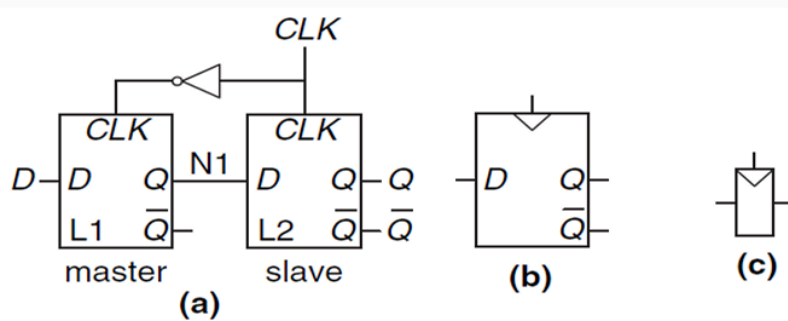




Advantages of D Latch over S-R Latch:

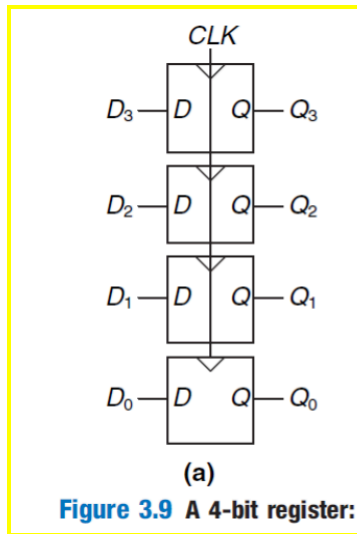
- The D latch is simpler than the S-R latch as it uses a single input (D), eliminating invalid states caused by both S and R being 1. It also includes a clock input (CLK), enabling output updates only at specific times, making it ideal for synchronous circuits.

4.



D flip-flop: (a) schematic, (b) symbol, (c) condensed symbol

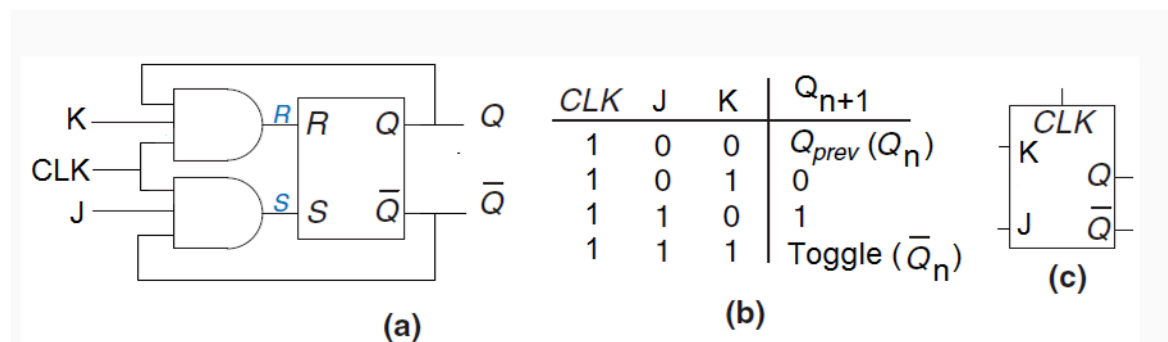
4-bit parallel in-parallel-out register



The importance of registers in a computer system

Registers are crucial in computer systems for temporary data storage, fast access, synchronization of data flow, parallel processing, and control logic implementation.

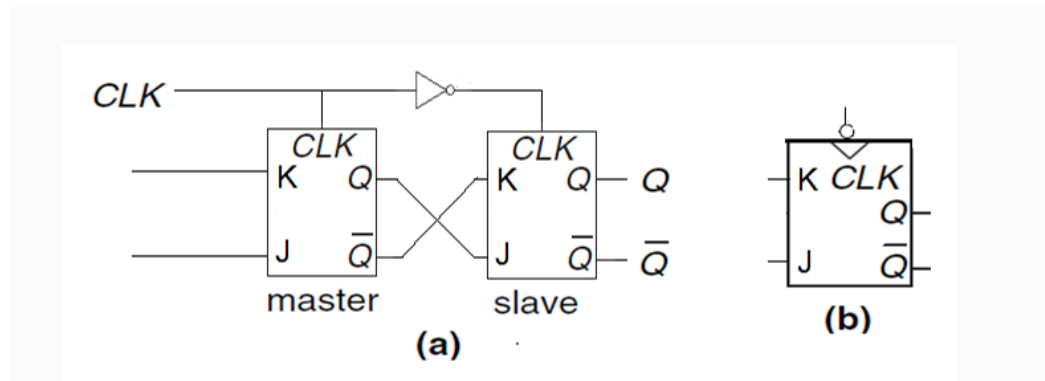
5. Designers eliminate the forbidden state of the S-R latch by creating a JK latch. This replaces S and R with J and K, allowing the output (Q) to toggle when both inputs are 1, preventing undefined states.



Truth Table

J	K	Q (Output)	$\bar{Q}$ (Complement)
0 ▾	0 ▾	Previous Q	Previous $\bar{Q}$
0 ▾	1 ▾	0	1
1 ▾	0 ▾	1	0
1 ▾	1 ▾	Toggle Q	Toggle $\bar{Q}$

6.J-K flip-flop



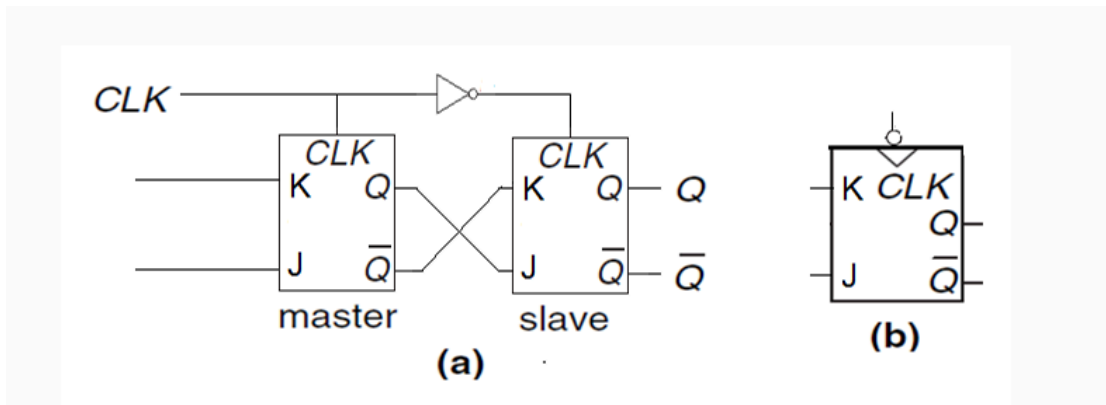
Truth Table of mod-13 ripple counter

State (Decimal)	Q3	Q2	Q1	Q0
0	0 ▾	0 ▾	0 ▾	0 ▾
1	0 ▾	0 ▾	0 ▾	1 ▾
2	0 ▾	0 ▾	1 ▾	0 ▾
3	0 ▾	0 ▾	1 ▾	1 ▾
4	0 ▾	1 ▾	0 ▾	0 ▾
5	0 ▾	1 ▾	0 ▾	1 ▾
6	0 ▾	1 ▾	1 ▾	0 ▾
7	0 ▾	1 ▾	1 ▾	1 ▾
8	1 ▾	0 ▾	0 ▾	0 ▾
9	1 ▾	0 ▾	0 ▾	1 ▾
10	1 ▾	0 ▾	1 ▾	0 ▾
11	1 ▾	0 ▾	1 ▾	1 ▾
12	1 ▾	1 ▾	0 ▾	0 ▾
13	0 ▾	0 ▾	0 ▾	0 ▾

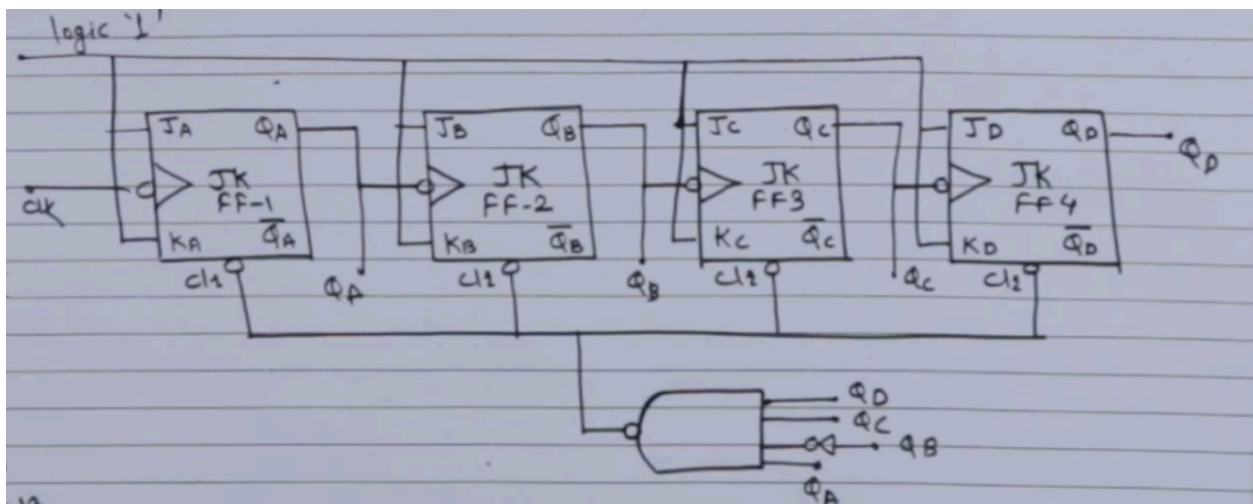
7. Toggling

Toggling in a J-K latch happens when $J = 1$ and $K = 1$, causing the output Q to alternate between 1 and 0 on each clock cycle

J-K flip-flop



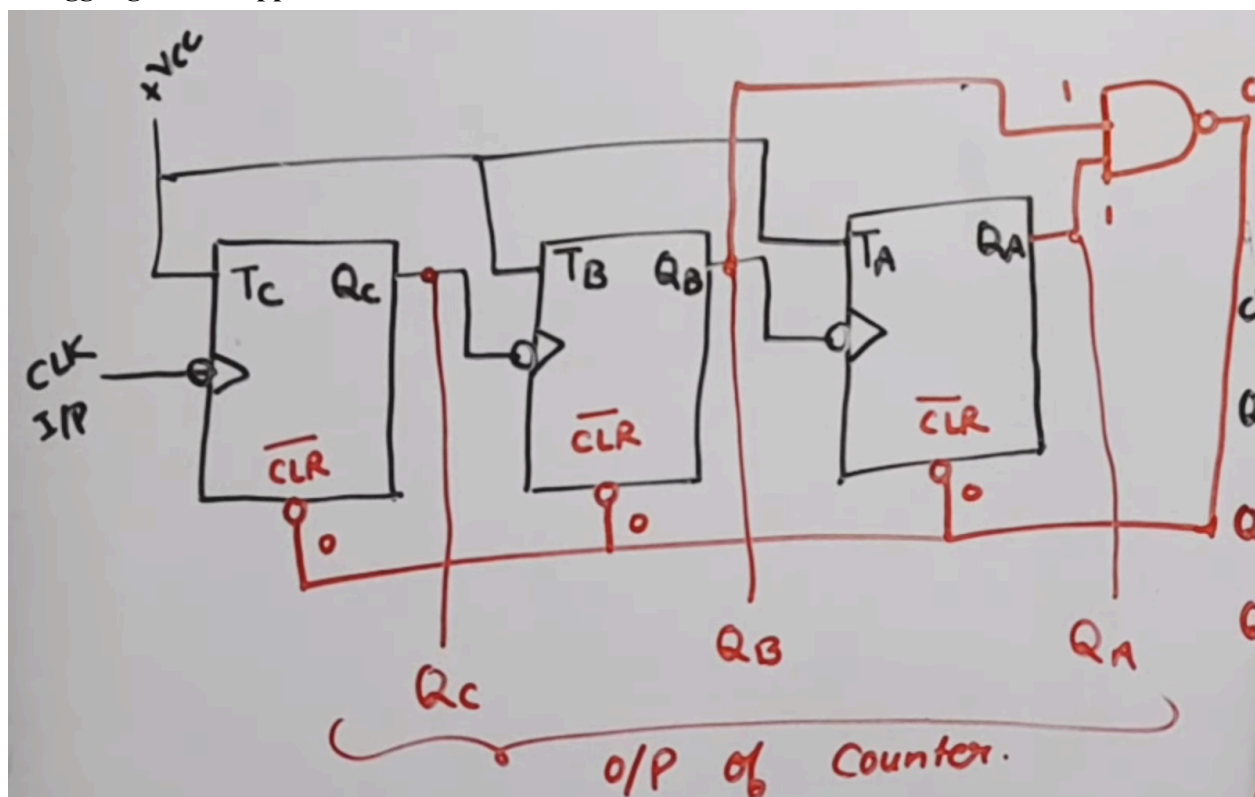
8. JK Flip Flop mod-13 ripple counter



State (Decimal)	Q3	Q2	Q1	Q0
0	0 ▾	0 ▾	0 ▾	0 ▾
1	0 ▾	0 ▾	0 ▾	1 ▾
2	0 ▾	0 ▾	1 ▾	0 ▾
3	0 ▾	0 ▾	1 ▾	1 ▾
4	0 ▾	1 ▾	0 ▾	0 ▾

5	0 ▾	1 ▾	0 ▾	1 ▾
6	0 ▾	1 ▾	1 ▾	0 ▾
7	0 ▾	1 ▾	1 ▾	1 ▾
8	1 ▾	0 ▾	0 ▾	0 ▾
9	1 ▾	0 ▾	0 ▾	1 ▾
10	1 ▾	0 ▾	1 ▾	0 ▾
11	1 ▾	0 ▾	1 ▾	1 ▾
12	1 ▾	1 ▾	0 ▾	0 ▾
13	0 ▾	0 ▾	0 ▾	0 ▾

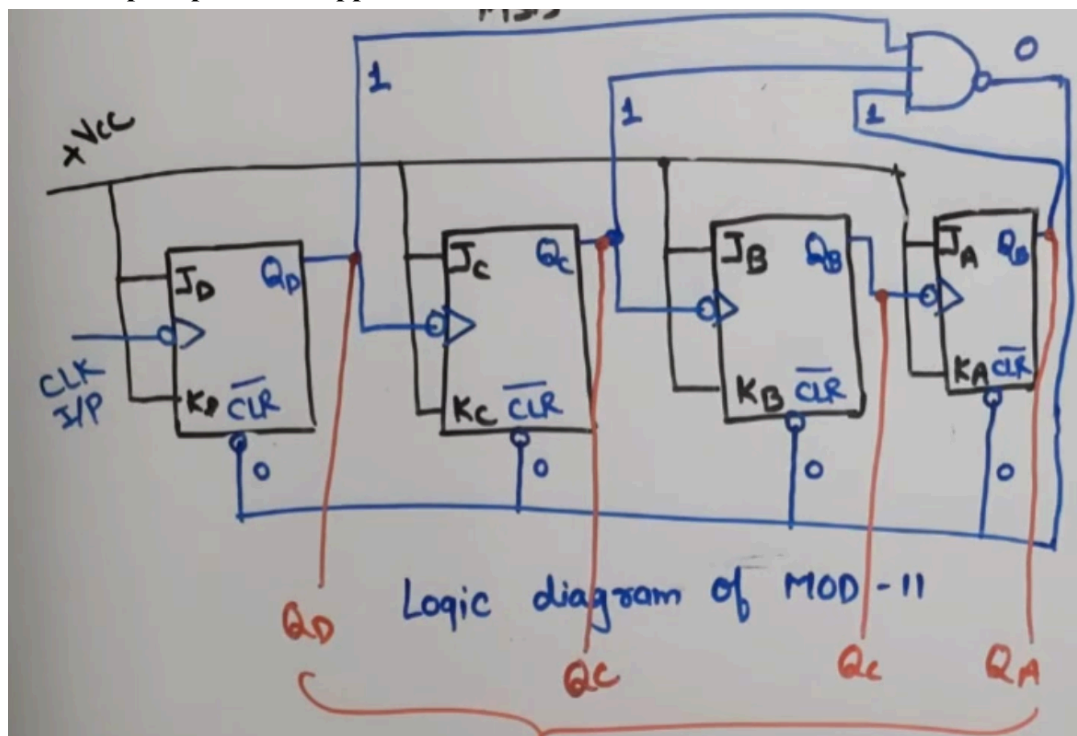
9. Toggling mod-6 ripple counter



State (Decimal)	Q3	Q2	Q1	Q0
-----------------	----	----	----	----

0	0 ▾	0 ▾	0 ▾	0 ▾
1	0 ▾	0 ▾	0 ▾	1 ▾
2	0 ▾	0 ▾	1 ▾	0 ▾
3	0 ▾	0 ▾	1 ▾	1 ▾
4	0 ▾	1 ▾	0 ▾	0 ▾
5	0 ▾	1 ▾	0 ▾	1 ▾
6	0 ▾	0 ▾	0 ▾	0 ▾

10. JK Flip Flop mod-11 ripple counter

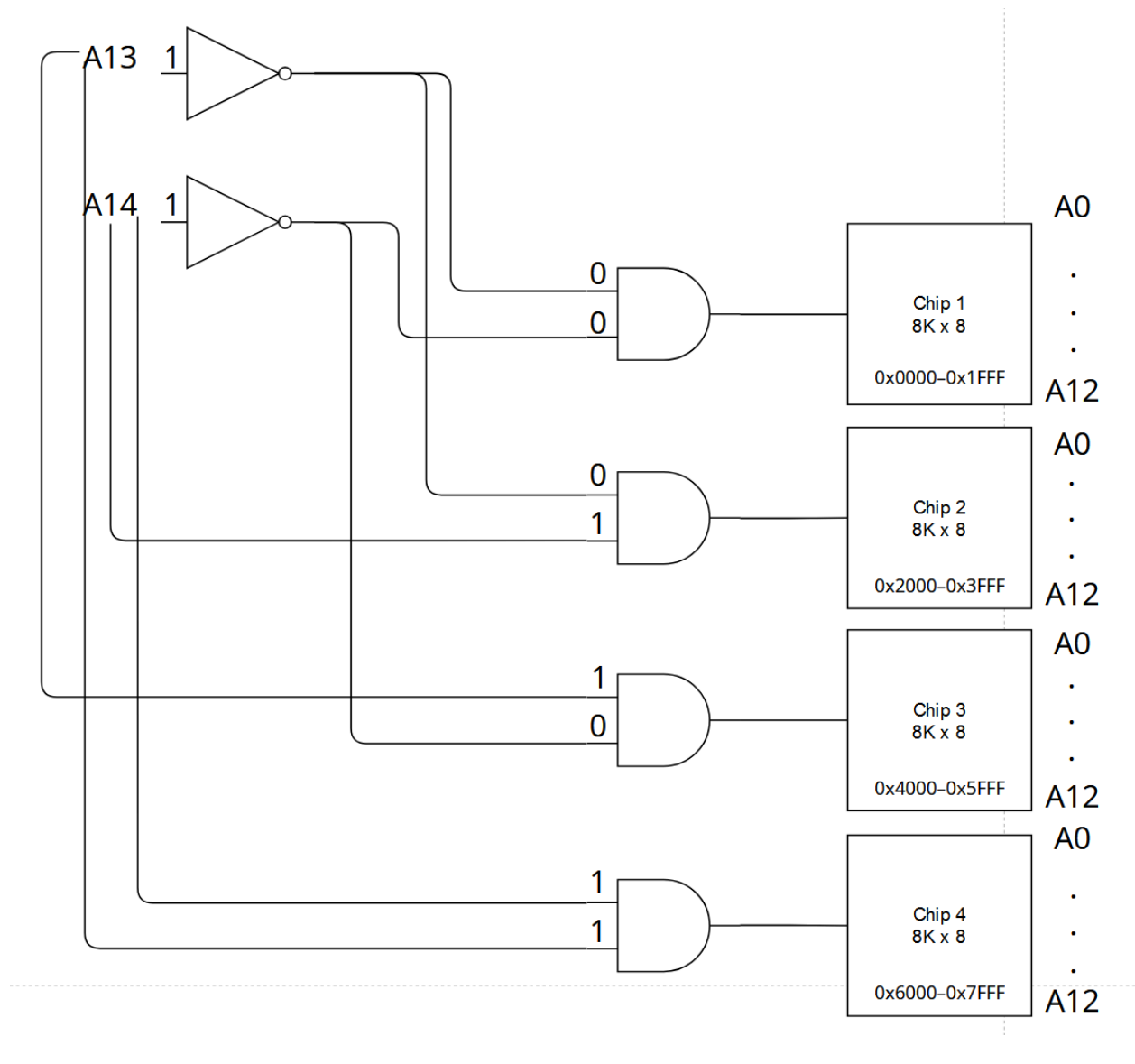


State (Decimal)	Q3	Q2	Q1	Q0
0	0 ▾	0 ▾	0 ▾	0 ▾
1	0 ▾	0 ▾	0 ▾	1 ▾
2	0 ▾	0 ▾	1 ▾	0 ▾

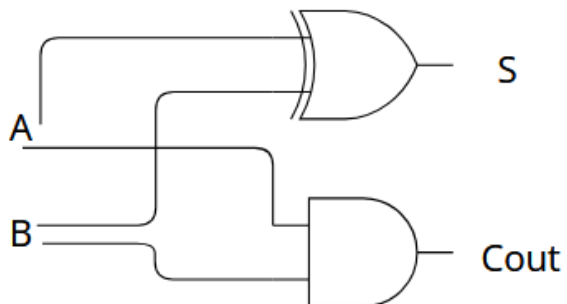
3	0 ▾	0 ▾	1 ▾	1 ▾
4	0 ▾	1 ▾	0 ▾	0 ▾
5	0 ▾	1 ▾	0 ▾	1 ▾
6	0 ▾	1 ▾	1 ▾	0 ▾
7	0 ▾	1 ▾	1 ▾	1 ▾
8	1 ▾	0 ▾	0 ▾	0 ▾
9	1 ▾	0 ▾	0 ▾	1 ▾
10	1 ▾	0 ▾	1 ▾	0 ▾
11	0 ▾	0 ▾	0 ▾	0 ▾

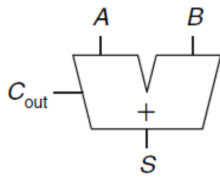
6530262_MinMyintMohSoe_Assignment(4)

1.4-line decoder and address ranges used by the decoder to select four memory chips (each with size 8K x 8).

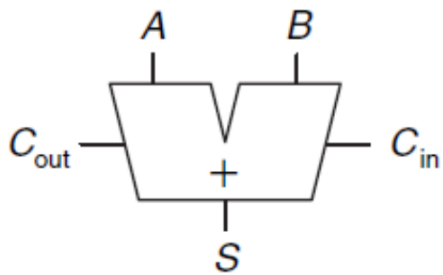
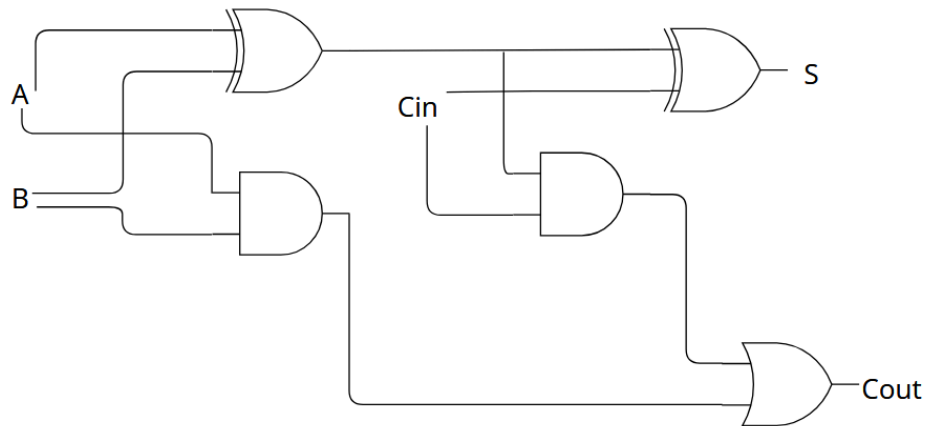


2. Circuit Schematic of Half Adder :

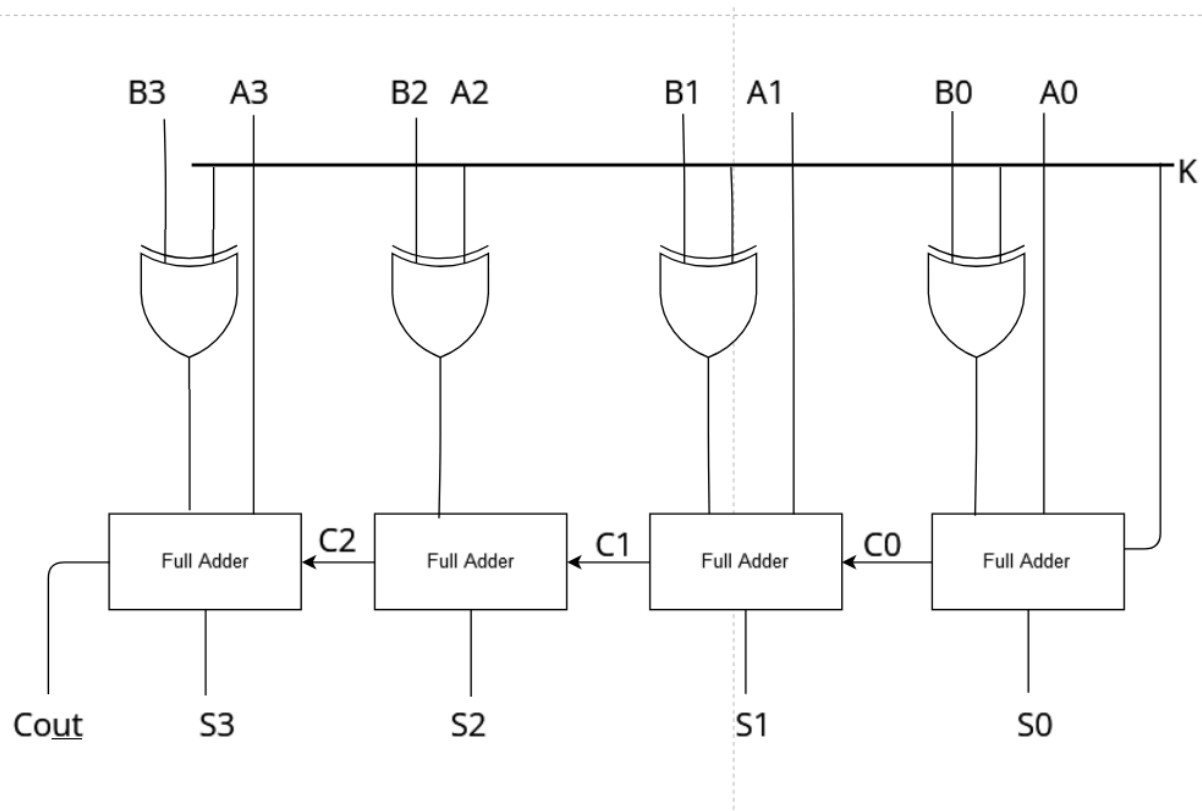




Circuit Schematic of Full Adder :



4-bit adder/subtractor circuit from the full adders



Addition

- Two 4-bit binary numbers A (A3, A2, A1, A0) and B (B3, B2, B1, B0) are inputs.
- The full adders add each pair of bits (A and B) bit by bit.
- Carry (C) is passed between adjacent adders.
- Sum Outputs: S3, S2, S1, S0 represent the result.
- Carry Out: Cout is the final carry after the last full adder.

Subtraction

- Subtraction is done by adding A and the two's complement of B.
- To get the two's complement of B:
 - Invert all bits of B (using XOR gates with a control input K).
 - Add 1 (control input K acts as a carry-in to the first adder).
- Control Input K:
 - K = 0: Performs addition.
 - K = 1: Performs subtraction (B is inverted, and 1 is added).

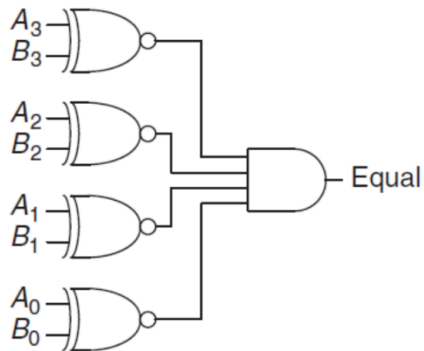
Result

- Addition: $S = A + B$.

- Subtraction: $S = A - B$ (using $A + \text{two's complement of } B$).

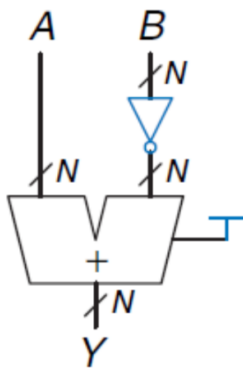
3.

4 bit Equality Comparator



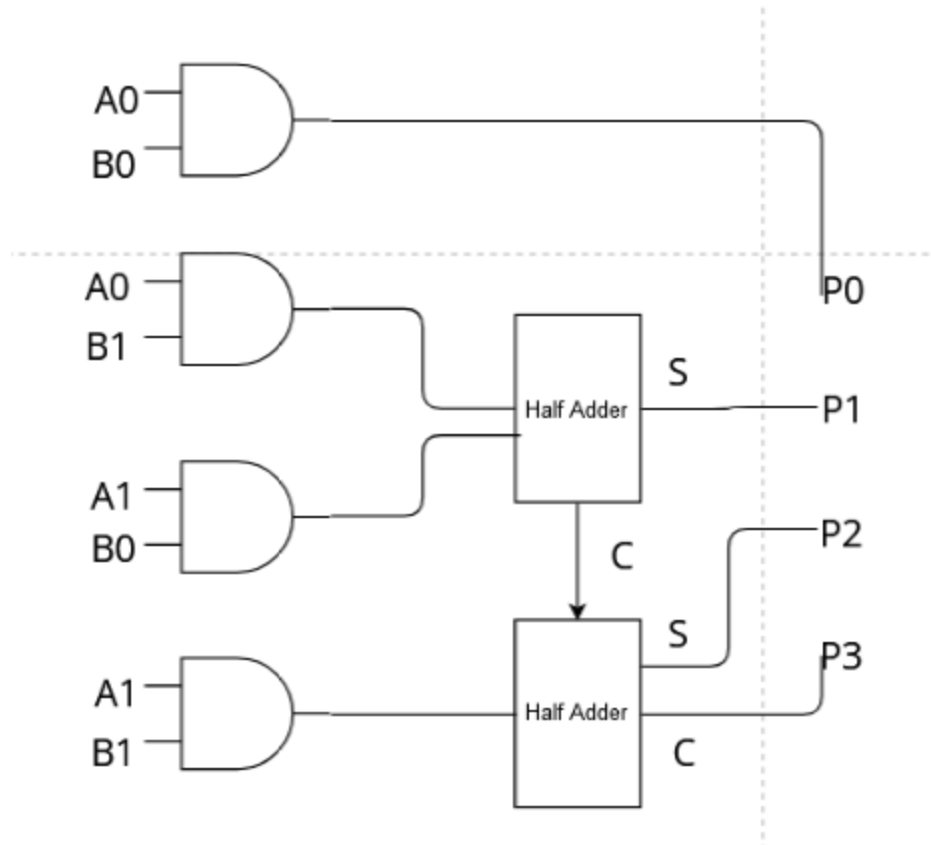
4-Bit Magnitude Comparator

The same circuit can compare two numbers, A and B, to determine:



1. $A > B$
2. $A < B$
 - Set $K = 1$ to subtract: $S = A - B$.
 - If the sign bit is 1, then $A < B$.
 - If the sign bit is 0, then $A \geq B$.

4.



Let's multiply two 2-bit binary numbers:

- $A = A1A0$ (where A1 is the MSB, A0 is the LSB).
- $B = B1B0$ (where B1 is the MSB, B0 is the LSB).

Take:

- $A = 10$ (binary) $\rightarrow A1 = 1, A0 = 0$.
- $B = 11$ (binary) $\rightarrow B1 = 1, B0 = 1$.

Step-by-Step Calculation

1. $P0$
 $P0 = A0 * B0$
 $P0 = 0 * 1 = 0$
2. $P1$
 $P1 = (A1 * B0) + (A0 * B1)$
 $P1 = (1 * 1) + (0 * 1)$
 $P1 = 1 + 0 = 1$

3. P2
 $P2 = (A1 * B1) + \text{Carry from previous addition}$
 $P2 = (1 * 1) + 0 \text{ (carry)}$
 $P2 = 1 + 0 = 1$
4. P3
 $P3 = \text{Final carry out}$
 $P3 = 0 \text{ (no additional carry)}$

Final Result

The 4-bit product is:

$P = P3P2P1P0 = 0110 \text{ (binary)}$

In decimal:

- $A = 2, B = 3$
- $A * B = 6$

5.

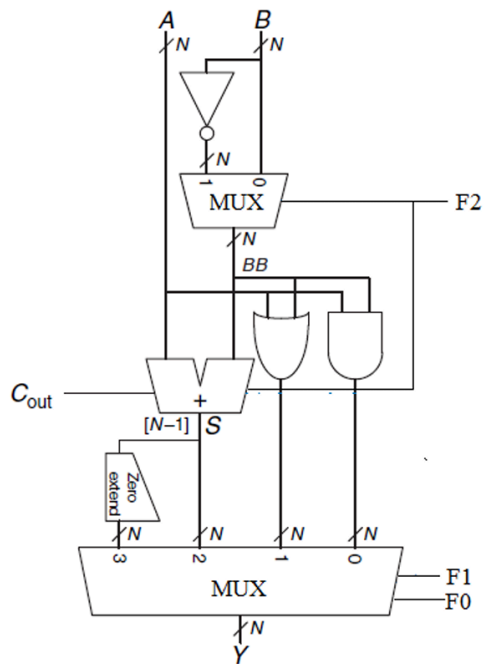


Figure 1.

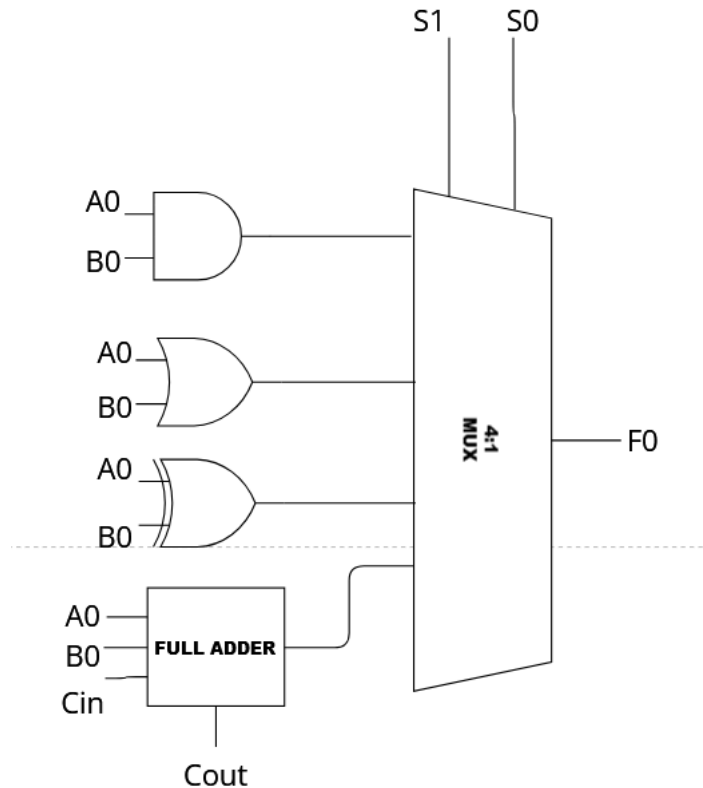
5.1 The circuit is an N-bit ALU (Arithmetic Logic Unit).

5.2: when $F2 = 0, F1 = 1$, and $F0 = 0$, $Y = A + B$

5.3: when $F2 = 1, F1 = 0$, and $F0 = 1$, $Y = A \text{ OR } B$

5.4: when $F2 = 1, F1 = 1$, and $F0 = 0$, $Y = A - B$

6. The schematics of an N-bit ALU (Arithmetic and Logic Unit) with four N bit functions such as addition, subtraction, OR operation, and parity checker :



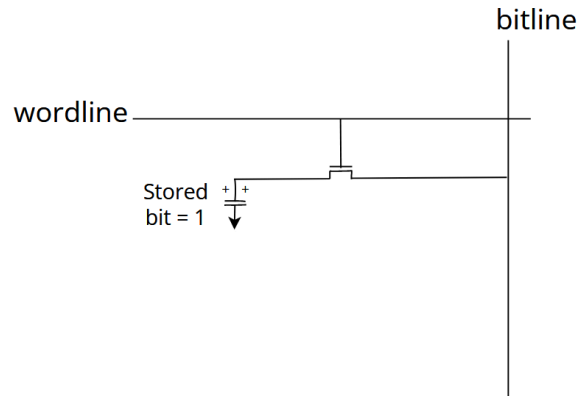
7.

Three main differences between SRAM (Static Random Access Memory) and DRAM (Dynamic Random Access Memory).

- **Storage Mechanism:**
 - **SRAM:** Stores data using cross-coupled inverters (latches), which are stable and do not require refreshing.
 - **DRAM:** Stores data as a charge on a capacitor, requiring periodic refreshing to retain the data.
- **Speed:**
 - **SRAM:** Faster access because it is built from latches, which directly output the stored value.
 - **DRAM:** Slower due to capacitor charge/discharge delay and periodic refresh cycles.
- **Cost and Size:**
 - **SRAM:** Expensive and requires more space.

- **DRAM:** Cheaper and more compact.

a 1-bit DRAM cell



Read Operation:

- When the wordline activates the transistor, the capacitor's charge is transferred to the bitline and read as either 1 (charged) or 0 (discharged).

Write Operation:

- The bitline is driven with the desired value (1 or 0).
- Activating the wordline allows the value to charge or discharge the capacitor accordingly

8. Maximum Address Bits:

For a 4 GB (4×10^9 bytes) main memory:

Memory is organized as $4 \times 1094 \times 10^9$ locations, each storing 1 byte (8 bits). To access each byte, the system needs to uniquely address all $4 \times 1094 \times 10^9$ locations. The required address bits N are given by $2^N \geq 4 \times 10^9$. $2^{32} = 4,294,967,296$ so 32 address bits are required.

Thus, the system uses 32 address bits.

9.

9.1. Read Only Memory (ROM):

Stores data permanently and is non-volatile (does not lose data when power is off).

Cannot be modified after manufacturing; mainly used for firmware.

9.2. Erasable Read Only Memory (EPROM):

Non-volatile memory that can be erased by exposing it to ultraviolet (UV) light.

Data can be rewritten after erasure.

9.3. EEPROM (Electrically Erasable Programmable ROM):

Non-volatile memory that can be electrically erased and reprogrammed without special equipment. More flexible than EPROM, but slower to write.

9.4. Flash Memory:

A type of EEPROM, but faster and cheaper because it erases data in larger blocks. Commonly used in USB drives, SD cards, and solid-state drives (SSDs)
